

File Access Os

While `ioutil` is great for small tasks (also deprecated), the **os package** is what professional Go developers use for "**precise control**." It allows you to read and write data in specific amounts (chunks), which is essential for performance and memory management.

1. Key Functions in the os Package

- **os.Open(filename)**: Opens an existing file for reading. It returns a **File Handle**.
- **os.Create(filename)**: Creates a new file (or truncates an existing one).
- **f.Read(byte_array)**: Reads data from the file to fill a specific byte array.
- **f.Write(byte_array)**: Writes the contents of a byte array into the file.
- **f.Close()**: Closes the file. **Always** close your files to prevent memory leaks!

2. Reading with Precision (The Buffer Method)

Unlike `ReadFile`, which forces you to take the whole file, **f.Read** lets you decide exactly how many bytes you want by providing a "**buffer**" (a byte array).

- **How it works**: If you create a byte array of size 10, Go will only pull the next 10 bytes from the file.
- **The "Read Head"**: Go keeps track of your position. If you call `Read` again, it picks up exactly where the last read finished.
- **The "Short Read"**: If your array is size 10 but the file only has 5 bytes left, the function returns the number of bytes actually read. (5 in this case)

```
f, _ := os.Open("data.txt")

// Create a buffer for 10 bytes
barr := make([]byte, 10)

// Read exactly 10 bytes into the buffer
nb, err := f.Read(barr)

fmt.Println(nb) // Prints how many bytes were actually read
f.Close()
```

3. Writing and Creating Files

The `os` package gives you two ways to put data into a file:

- **f.Write(byte_array)**: Best when you are working with raw binary data or bytes.
- **f.WriteString("string")**: A helper function that lets you write text directly without manually converting it to bytes first.

```
f, _ := os.Create("outfile.txt")

// Method 1: Writing a slice of bytes
data := []byte{1, 2, 3}
f.Write(data)

// Method 2: Writing a string directly
f.WriteString("Hi there!")

f.Close()
```

Expert Takeaway

In backend engineering, you almost never want to read a file "all at once" if you don't know how big it is. Using the `os` package with a **buffer** (usually 4KB or 8KB in size) allows you to process massive files (like gigabyte-sized logs) using only a tiny amount of RAM.

Pro Tip: In Go, we use the **defer** keyword to ensure files are closed even if our code crashes. You will almost always see: `f, _ := os.Open("file.txt") defer f.Close()` This tells Go: "No matter what happens, close this file before this function finishes."